

Lab 06 – Seven Segment Decoder Design

Name: sualeh abbas	ID:	Section:
--------------------	-----	----------

Objectives

After performing this lab, student should be able to:

- Design Verilog modules from given truth table
- Test the functionality of design module through a testbench
- Design Verilog modules from sub-modules using hierarchy.

This lab consists of two different sections. The description of each section is given below:

Section	
a) <u>Introduction</u>  In this lab, you're going to build a system that is capable of adding and subtracting two 3-bit binary numbers.	30
b) <u>7-Segment Hexadecimal Display Decoder</u>  In this section, you will construct your own seven-segment display decoder circuit using Verilog HDL which will have four input bits (D0 to D3), representing a binary coded hexadecimal number between 0 and F and produce seven output bits (Sa to Sg).	100
c) <u>Adder for 3bit binary numbers</u>  In this part, you will design a circuit that functions as an adder for 3-bit binary numbers. You will also simulate and verify the functionality of a hardware developed, using HDL, using Xilinx Vivado.	30



a. Introduction

In lab 06, 07 and 08, you're going to build a system that is capable of adding and subtracting two 3-bit binary numbers, A and B. The system accepts A, B, and an operator bit, O, which can either be 0 or 1, as input. If the bit O is 0 then the system adds A and B, and when O is 1 it subtracts B from A. In either case, the system generates the appropriate signals for seven-segment display to show the result of addition or subtraction as a hexadecimal digit on Basys-3 board. If the result is negative then the dp light of seven segment display is used to indicate that the displayed number is a negative number. The block diagram of the complete system is shown below. Before moving to the design, try to convince yourself that system given in Figure 6.0 works as per procedure for addition and subtraction of any 3-bit binary numbers. Try 3-2 (i.e., 011-010) and 2-3 (010 -011) and consider more test cases of your choice.

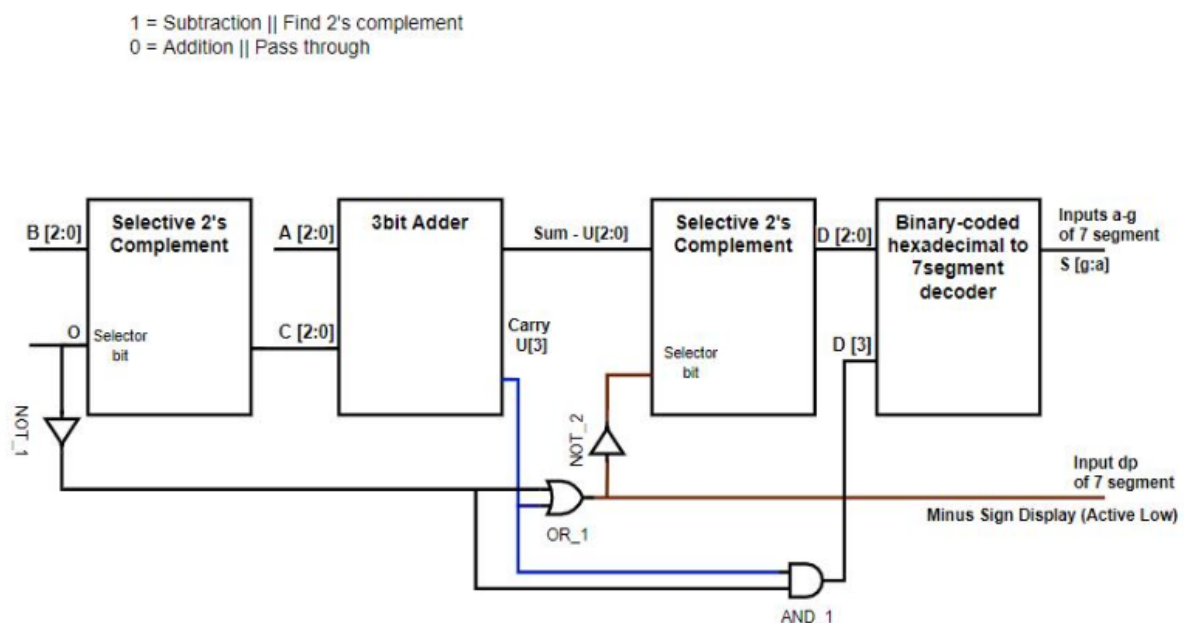


Figure 6.0: Add/Subtract Module

This lab is divided in three parts, in Lab 06, you will design and test the 3-bit adder module and seven segment decoders. In Lab 07, you will design 2's complement module followed by combining all the designed modules as per above figure. In Lab 08, you will implement the design on BASYS-3 and see it in action. For all labs, beside designing design module, you will design testbench modules as well as you will need to test each design thoroughly before integration. A worksheet is shared for each lab, which you will use to report your results.

b. 7-Segment Hexadecimal Display Decoder

In a previous lab, you became familiar with displaying numbers on a seven-segment display using a 7447 IC, which is a BCD to seven-segment decoder. It is called a decoder because it takes the four inputs (a BCD) and decodes them into seven outputs, which are then used to drive one or more of the seven segments of the seven-segment display to show the desired number. Your task in this lab is to construct your own seven-segment display decoder, which will replace the 7447 IC. But you'll be designing a slightly different seven-segment display decoder than 7447 IC, as shown in Figure 6.1. This decoder circuit will have four input bits (D_0 to D_3), representing a binary coded hexadecimal number between 0 and F, and produce



seven output bits (S_a to S_g), which will be used to light up the appropriate segments of the seven-segment display according to Figure 6.2.

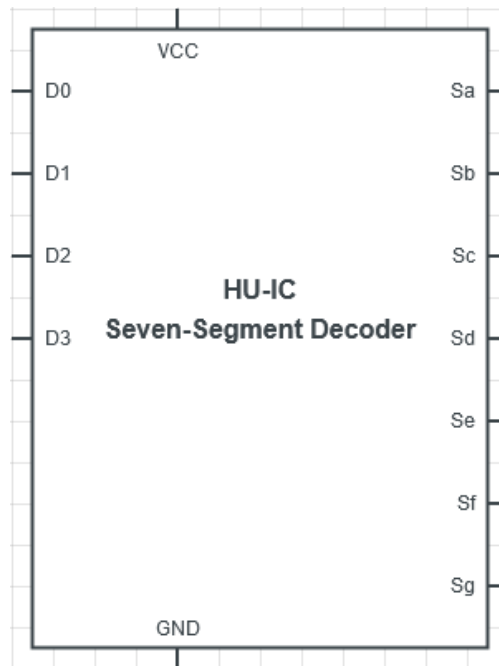


Figure 6.1: Seven segment hexadecimal display decoder

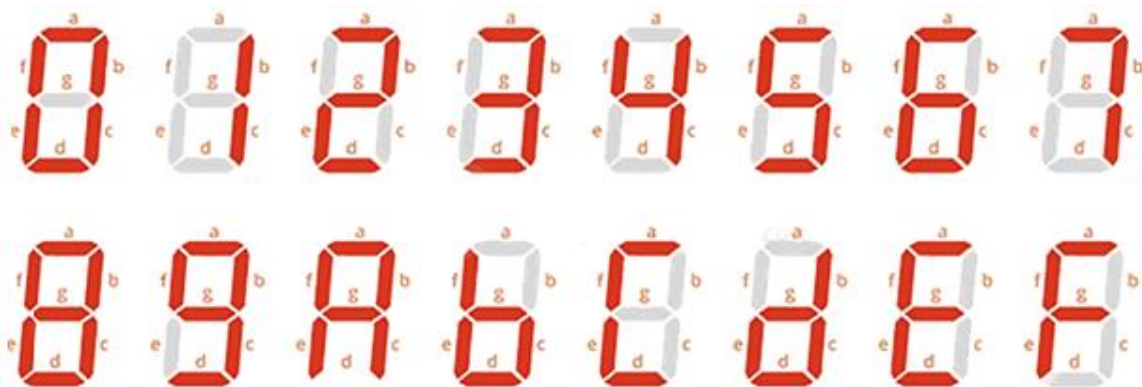


Figure 6.2: Segment scheme for hexadecimal display

To design your **Common-Anode** seven-segment display decoder you will first write the truth table specifying the output values for each input combination. Complete the truth table provided in Table 6.1. As an example, when the input is $D_{3:0}=0000$, i.e. we want to display zero, then all of the segments except g should be on. Because the outputs are active low, they must be $S_{g:a}=1000000$. As you may have noticed, **active low** implies that a segment is active when a 0 or LOW voltage is provided to it.



Table 6.1

Hex Digit	Binary-coded Hexadecimal/ Inputs to decoder				Outputs of the decoder						
	D ₃	D ₂	D ₁	D ₀	S _g	S _f	S _e	S _d	S _c	S _b	S _a
0	0	0	0	0	1	0	0	0	0	0	0
1	0	0	0	1	1	1	1	1	0	0	1
2	0	0	1	0	0	1	0	0	1	0	0
3	0	0	1	1	0	1	1	0	0	0	0
4	0	1	0	0	0	0	1	1	0	0	1
5	0	1	0	1	0	0	1	0	0	1	0
6	0	1	1	0	0	0	0	0	0	0	1
7	0	1	1	1	1	1	1	1	0	0	0
8	1	0	0	0	0	0	0	0	0	0	0
9	1	0	0	1	0	0	1	0	0	0	0
A	1	0	1	0	0	0	0	1	0	0	0
B	1	0	1	1	0	0	0	0	0	1	1
C	1	1	0	0	1	0	0	0	1	1	0
D	1	1	0	1	0	1	0	0	0	0	1
E	1	1	1	0	0	0	0	0	1	1	0
F	1	1	1	1	0	0	0	1	1	1	0

Write Boolean expression for each output segment in Table 6.2. You should have seven expressions for S_a through S_g. Each expression will be in terms of inputs D₀ to D₃ and will hold for the corresponding column in the truth table above, i.e., if you have written an expression for S_a, then for each row combination of D₀-D₃ in the above table, the expression will output 1/0 according to entry in the column S_a. It's a good idea to simplify these expressions, before we implement them. Try simplifying a few expressions by hand or use online Boolean expression simplification tool (<http://www.32x8.com/index.html>) for the rest. Once you learn of K-maps in the coming weeks, you'll be able to directly obtain simplified expressions from truth tables.

S _a =	$\sim D_3 \& D_2 \& \sim D_0 + \sim D_3 \& \sim D_2 \& \sim D_1 \& D_0 + D_3 \sim D_2 D_1 D_0 + D_3 D_2 \sim D_1 D_0$
S _b =	$ACD + AB \sim D + \sim AB \sim CD$
S _c =	$AB \sim D + ABC + \sim A \sim BC \sim D$
S _d =	$BCD + \sim A \sim B \sim CD + \sim AB \sim C \sim D + A \sim BC \sim D$
S _e =	$\sim AD + \sim B \sim CD + \sim AB \sim C$
S _f =	$\sim A \sim BD + \sim A \sim BC + \sim ACD + AB \sim CD$
S _g =	$\sim A \sim B \sim C + \sim ABCD + AB \sim C \sim D$

Table 6.2





Write a Verilog module for your decoder that implements the above expressions in Xilinx Vivado. The module definition is as follows (Assume that $S_{g:a} = S_{6:0}$).

```
module segmentDecoder(D,S);
    input [3:0] D;
    output [6:0] S;
    // Write your code here
    ...
    ...
endmodule
```



It is possible to address bits or parts of a vector. For example, $D[3]$ refers to the MSB of D .

As you learnt in the previous lab, every design is to be thoroughly tested after it has completed, which requires the creation of a test bench. The testbench for the decoder has already been created for you. Copy the code from the provided file on Canvas into Xilinx Vivado. The code has also been included here for reference.

```
module segmentDecoderTest();
    reg [3:0] dD;
    wire [6:0] dS;

    //Instantiating the unit under test (UUT)
    segmentDecoder I1(dD,dS);

    // Stimulus
    initial
        begin
            dD = 4'h0;
            #10 dD = 4'h1; // h represents hexadecimal
            #10 dD = 4'h2;
            #10 dD = 4'h3;
            #10 dD = 4'h4;
            #10 dD = 4'h5;
            #10 dD = 4'h6;
            #10 dD = 4'h7;
            #10 dD = 4'h8;
            #10 dD = 4'h9;
            #10 dD = 4'hA;
            #10 dD = 4'hB;
            #10 dD = 4'hC;
            #10 dD = 4'hD;
            #10 dD = 4'hE;
            #10 dD = 4'hF;

            $monitor("Time=%0d, D=%4b. S=%7b\n", $time, dD, dS );
        end
endmodule
```



An initial block can contain sequential statements that can be used to describe the behavior of signals in a test bench.



#10 adds a delay of 10 units of time in the sequential execution. This is units of simulator time and not 10ns or 10ps.



\$monitor is a system task similar to **\$display**. While **\$display** statement displays content only when it is called, **\$monitor** displays the result every time one of its specified parameters changes.
\$time returns the current simulation time.

Appropriately add comments to your code and **submit** the code of your design module, attach screenshot of the RTL Schematic of the module, simulation output window and logs-window in worksheet.

For your reference for lab 8, the decoder circuit that you have just designed and verified can be used to drive a common-anode seven segment display according to Figure 6.3, replacing the seven-segment decoder block with your own designs. The resistor for each segment of the display limits the current flowing through that segment so that it is not damaged.

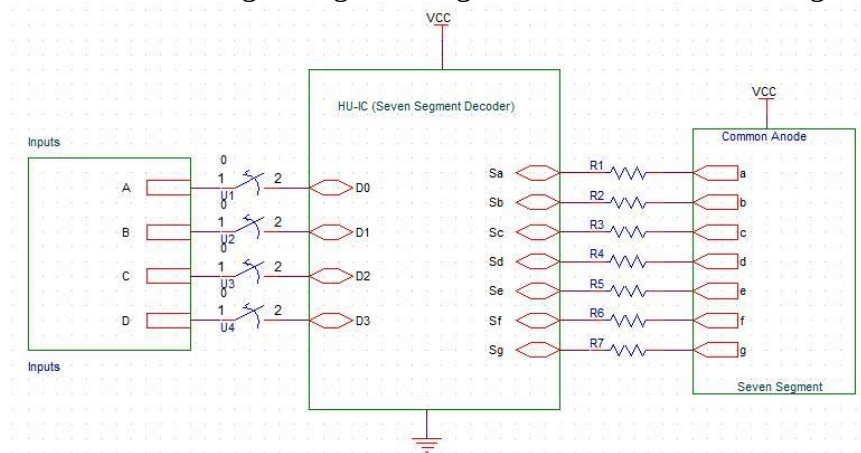


Figure 6.3 Seven Segment Decoder Circuit

c. Adder for 3-bit binary numbers

Using the full adder module from Lab 05 exercise 03 and following the cascading approach of lab 04, design a logic module that functions as an adder for 3-bit binary numbers. Note that this adder module will have six input bits i.e., 3 bits corresponding to one number (A) and 3 bits to the other (B), and will generate four output bits, i.e., the three sum bits and a carry-out bit.



Reference Design Module	Reference Testbench Module
<pre> `timescale 1ns/1ps //Three bit full adder or Ripple Carry Adder module ThreebitFA(input [2:0] A, input [2:0] B, output [2:0] S, output Cout); wire Cout1,Cout2; Fulladder FA1 (A[0],B[0],0,S[0],Cout1); Fulladder FA2 (A[1],B[1],Cout1,S[1],Cout2) ; Fulladder FA3 (A[2],B[2],Cout2,S[2],Cout); endmodule //Full Adder module Fulladder(input a, input b, input c_in, output sum, output c_out); assign sum=a^b^c_in; assign c_out=(a&b) (a&c_in) (b&c_in); endmodule </pre>	<pre> module tb(); reg [2:0] A; reg [2:0] B; wire [2:0] S; wire Cout; ThreebitFA tesetmodule (A,B,S,Cout) ; initial begin A=3'b000; B=3'b000; #10 A=3'b111; B=3'b101; end endmodule </pre>



Verilog allows you to create hierarchical abstractions, so you can create multiple modules within a single design and have instantiations of one module in another.

Assessment Rubrics

Marks Distribution:

		LR2 Code	LR4 Data Collection	LR5 Results	LR7 Viva	LR9 Report
In-lab	Task a				10 points	10 points
	Task b	15 points	20 points	15 points		
	Task c	20 points		10 points		
Total Marks =		100 points				

Marks Obtained:

		LR2 Code	LR4 Data Collection	LR5 Results	LR7 Viva	LR9 Report
In-lab	Task a	-	-	-		
	Task b					-
	Task c		-			-
Total Marks =						

Lab Evaluation Rubrics

#	Assessment Elements	Level 1: Unsatisfactory	Level 2: Developing	Level 3: Good	Level 4: Exemplary
LR2	Program/Code/Simulation Model/Network Model	Program/code/simulation model/network model does not implement the required functionality and has several errors. The student is not able to utilize even the basic tools of the software.	Program/code/simulation model/network model has some errors and does not produce completely accurate results. Student has limited command on the basic tools of the software.	Program/code/simulation model/network model gives correct output but not efficiently implemented or implemented by computationally complex routine.	Program/code/simulation model/network model is efficiently implemented and gives correct output. Student has full command on the basic tools of the software.
LR4	Data Collection	Measurements are incomplete, inaccurate and imprecise. Observations are incomplete or not included. Symbols, units and significant figures are not included.	Measurements are somewhat inaccurate and imprecise. Observations are incomplete or vague. Major errors are there in using symbols, units and significant digits.	Measurements are mostly accurate. Observations are generally complete. Minor errors are present in using symbols, units and significant digits.	Measurements are both accurate and precise. Data collection is systematic. Observations are very thorough and include appropriate symbols, units and significant digits and task completed in due time.
LR5	Results & Plots	Figures/ graphs / tables are not developed or are poorly constructed with erroneous results. Titles, captions, units are not mentioned. Data is presented in an obscure manner.	Figures, graphs and tables are drawn but contain errors. Titles, captions, units are not accurate. Data presentation is not too clear.	All figures, graphs, tables are correctly drawn but contain minor errors or some of the details are missing.	Figures / graphs / tables are correctly drawn and appropriate titles/captions and proper units are mentioned. Data presentation is systematic.
LR7	Viva	Response shows a complete lack of understanding of the assigned / completed task	Response shows shallow understanding of the assigned task	Response shows substantial understanding of the assigned task	Response shows complete understanding of the completed task. The student is able to explain all the related concepts.
LR9	Report	No summary provided. The number/amount of tasks completed below the level of satisfaction and/or submitted late	Couldn't provide good summary of in-lab tasks. Some major tasks were completed but not explained well. Submission on time. Some major plots and figures provided	Good summary of In-lab tasks. All major tasks completed except few minor ones. The work is supported by some decent explanations, Submission on time, All necessary plots, and figures provided	Outstanding Summary of In-Lab tasks. All task completed and explained well, submitted on time, good presentation of plots and figure with proper label, titles and captions